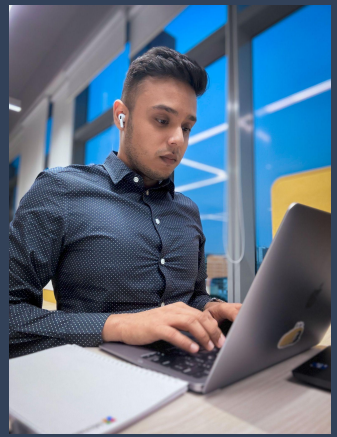
Agentic AI Foundations

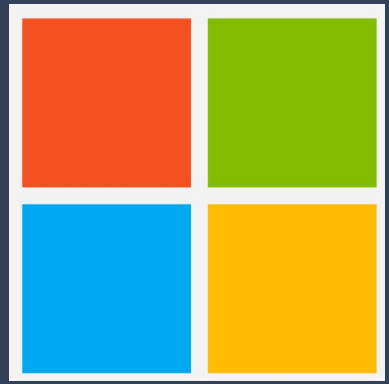


Anshaj Khare

Introduction – Anshaj Khare



- ✔ I have been working on data science since **2017**
- ✔ I've worked for 4 startups before joining Microsoft
- ✔ I have been working at Microsoft for 4+ years as a ML Engineer
- ✔ In my career, I've worked as a backend engineer, a devops engineer, a data engineer and as a Machine learning engineer
- ✔ Have taught over 4000+ learners - Python, Data Science and Gen AI
- ✔ Author: https//compute.anshaj.dev/
- ✔ Github: http://github.com/anshajk



{ik} INTERVIEW KICKSTART

Welcome to today's class, before we begin, pop into the chat:

Your Name



Role

Location

Company

{ik} INTERVIEW KICKSTART

Optimize Your Experience



- ✅ Interact with your instructors via Sunday live class.
- ✅ Don’t be shy to speak up and get clarifications !



- ✅ Solve your assignments, MCQs and other assessments and get feedback (Saturday review session)
- ✅ Use your resources! It’s your experience – what you put in is what you’ll get out.

{ik} INTERVIEW KICKSTART

Before we start / How we got here

1970s – 1990s



Rule-Based Systems

If-then logic, expert systems, hand-coded rules.

Brittle, hard to maintain, couldn't handle the unexpected.

2000s – 2010s



Machine Learning

Learn patterns from data. Classification, prediction, recs.

Powerful but narrow one model <-> one task.

2020s



Generative AI

Generates text, code, images. General-purpose & conversational.

{ik} | INTERVIEW KICKSTART

And now GenAI is changing everything...



How

we build software

- AI-assisted coding & code review
- AI-driven testing & debugging
- AIOps -agents handling ops tasks

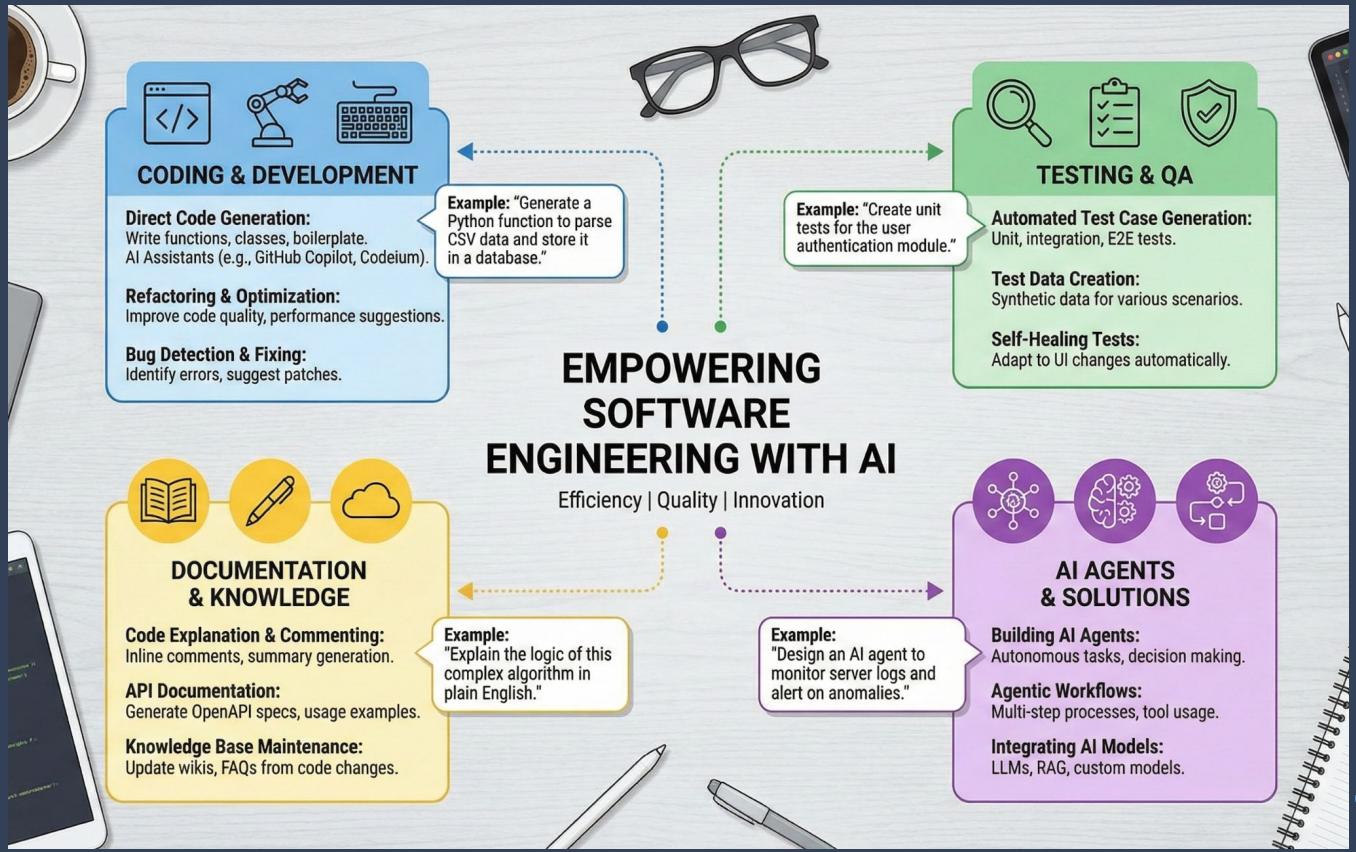


What

software we build

- AI-native products & features
- Conversational interfaces
- Agent-powered workflows

Before we begin...



{ik} INTERVIEW KICKSTART

From AI Tools to Agentic Systems: The Shift in Software Development

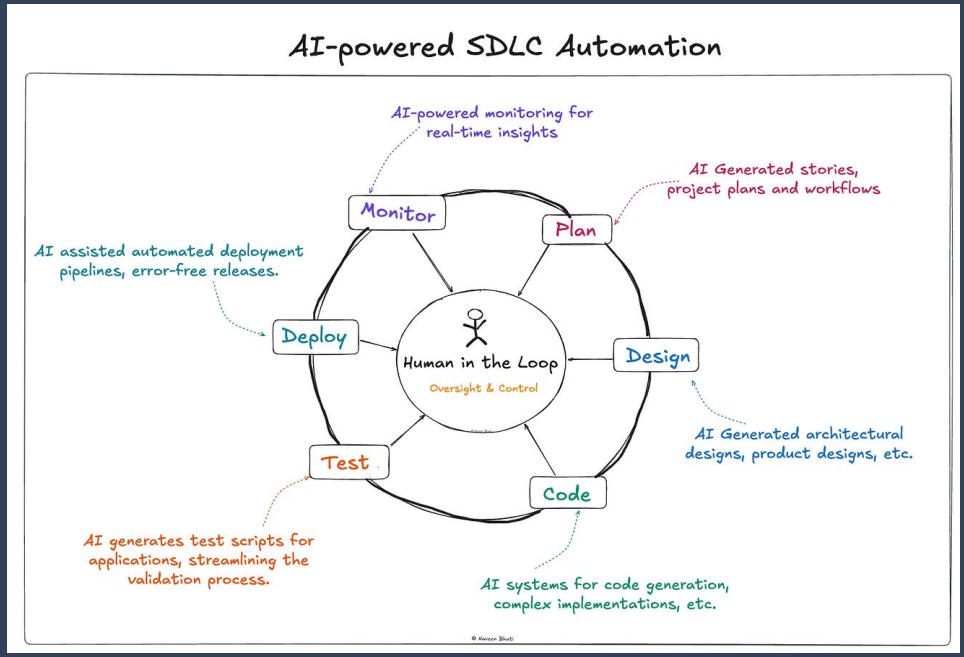
- Agentic AI is shifting software development from **tools that assist developers** to **systems that execute goals autonomously**.
- Instead of responding to single prompts, agents plan, decide, act, observe outcomes, and iterate.
- **The Shift Driven by Agentic AI:**
    - From static code → adaptive systems
    - From function calls → decision-driven workflows
    - From manual orchestration → autonomous execution

{ik} INTERVIEW KICKSTART

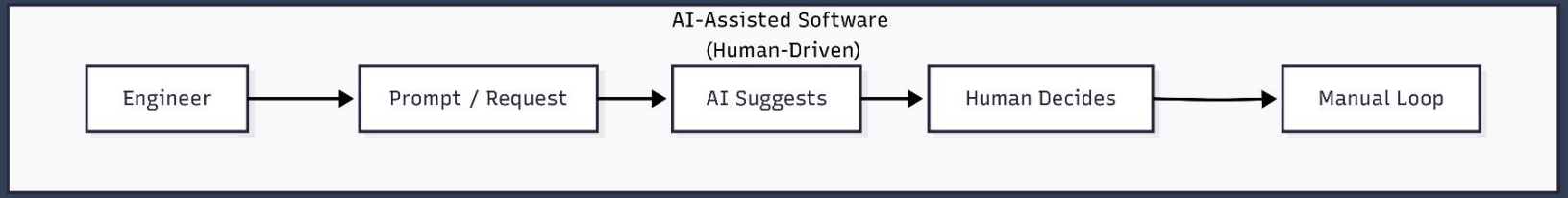
From AI Tools to Agentic Systems: The Shift in Software Development

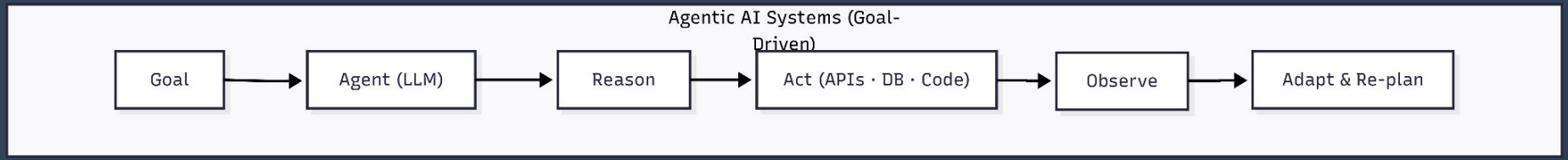
The Entire SDLC Is Turning Into an AI-Driven Pipeline

- AI plans requirements
- Builds multi-file features
- Runs tests automatically
- Deploys & monitors continuously



{ik} INTERVIEW KICKSTART



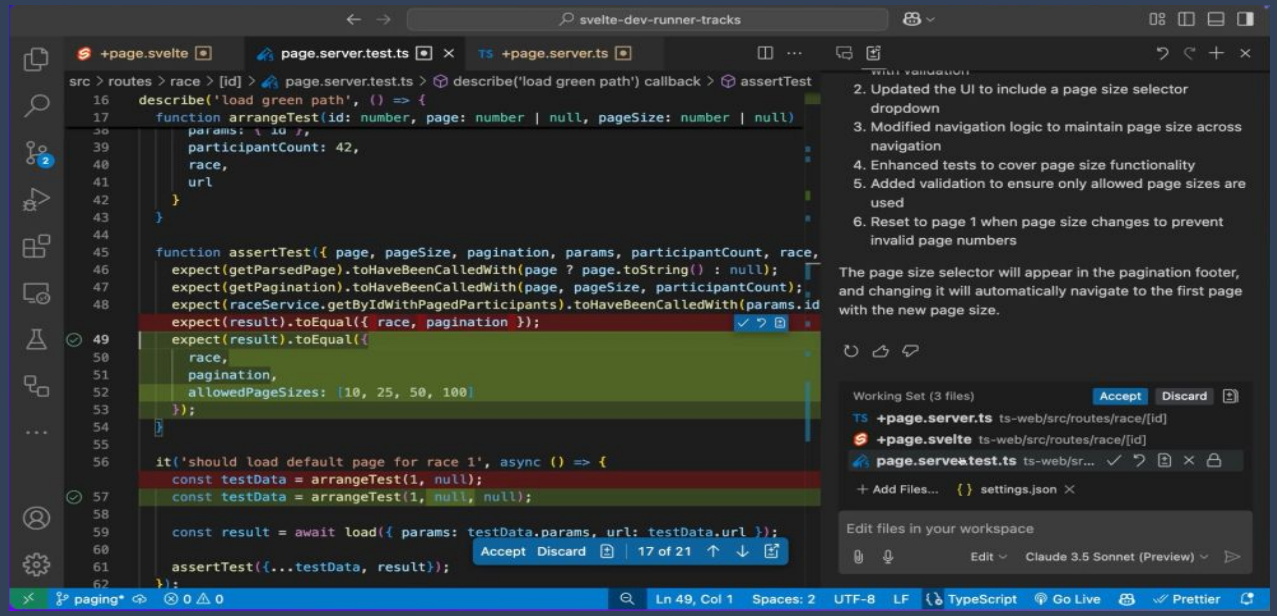


With execution responsibility delegated to agents, the focus shifts to what they can automate.

{ik} INTERVIEW KICKSTART

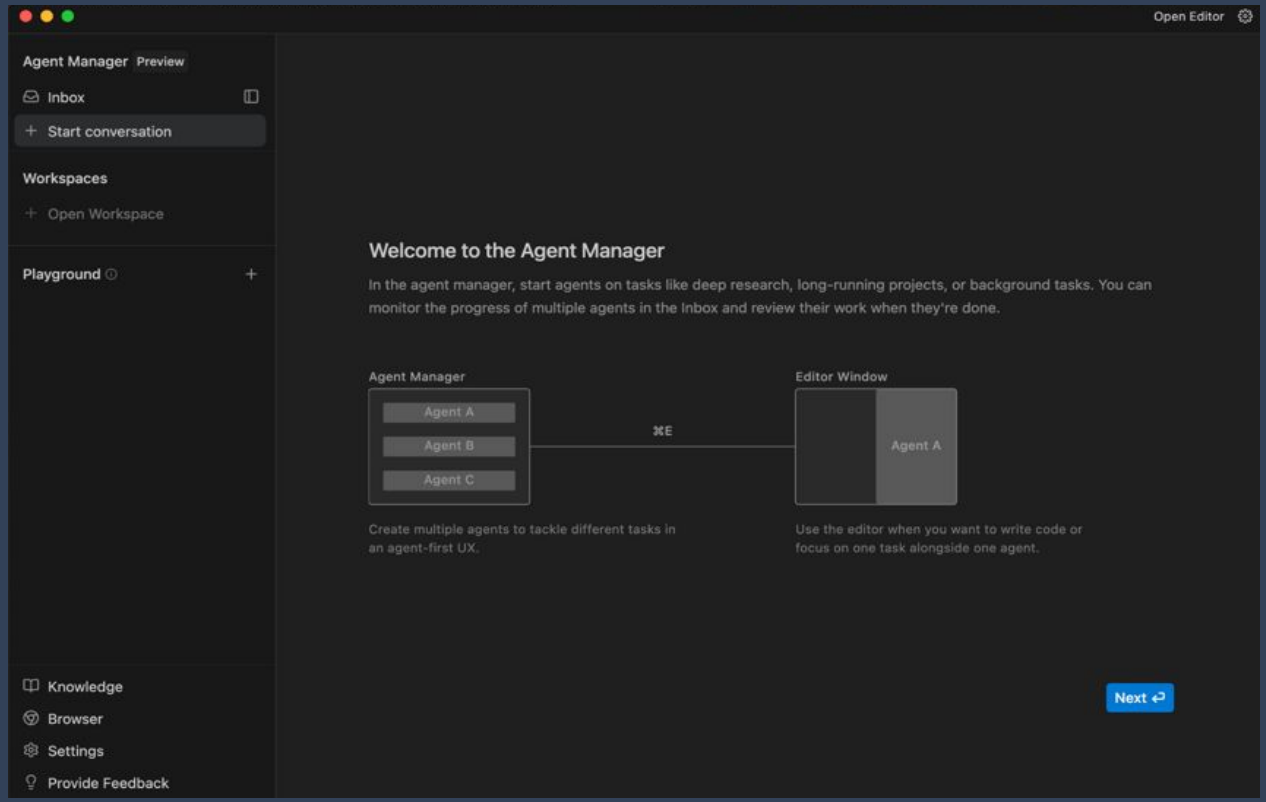
What Kind of AI Automations Are We Looking At?

- GitHub Copilot agent mode that writes code autonomously, detects errors in real time, runs tests, and creates pull requests for review.



{ik} INTERVIEW KICKSTART

- Google Antigravity IDE where multiple AI agents work in the editor and terminal to build apps, test code, and manage workflows inside a development environment.



{ik} INTERVIEW KICKSTART

Before we dive into Agents, let's take a quick look at what this course will cover.

{ik} INTERVIEW KICKSTART

Agentic AI Program Structure

Foundations of Agentic AI Systems	Intelligent Interaction & Multi-Agent Orchestration	Real-World Intelligence & Smart Automation	Optimization, Safety & Enterprise Systems
0: Pre-Program Foundations • Python essentials & Hello Agent	4: Multi-Agent Systems • Planner-Executor-Critic patterns, orchestration frameworks, cost/latency tracking • Project: AI Research Team	7: Domain-Specific Vertical Agents: Summarization & Recommendation Agents • API integration & OAuth, structured JSON workflows, hybrid RAG + live API systems • Text summarizers, ranking/ recommendation systems, analytics dashboards	8: Safe Agents • Guardrails & validation layers, LangSmith/TruLens observability, cost monitoring • Project: Safe Support Agent
Week 0,1, 2 & 3	5: Conversational Agents • Memory systems (short/long-term), voice I/O integration, multimodal pipelines	• Project: Smart Recommender	9: Fine-Tuning • LoRA/PEFT techniques, custom training pipelines, model deployment strategies • Project: Fine-Tuned Domain Agent
1: Agentic AI Foundations • Reflex/goal/utility agents, modular architecture, tool calling logic • Project: Reflex FAQ Agent	*Week 4,5 & 6*	*Week 7*	*Week 8, 9, 10 & 11*
2 & 3: Knowledge Agents (RAG) • Embeddings & chunking strategies FAISS/Chroma vector stores, document ingestion pipelines • Project: Knowledge Assistant	• Project: Research Assistant 6: Agent Protocols • MCP/A2A/ACP standards, LangGraph workflow design, async messaging patterns • Project: Negotiation Simulator		10–11: Capstone System • End-to-end multi-agent architecture, production monitoring, deployment to cloud

VIEW KICKSTART

By the end of this course, you will be able to:

- Build and deploy agents: from simple agents to multi-agent systems
- Design agentic systems with real-world constraints and tradeoffs
- Integrate agents into your existing tools and workflows
- Ship production-ready agents with observability & evaluation
- Fine-tune and tailor agentic systems for complex, domain-specific tasks

What You've Been Doing So Far: LLMs & Functions

The Shift Toward Agentic Systems

- **The Hidden Shift:**
  - LLMs + functions = daily workflow integration
  - Intelligence + execution working together
  - Foundation for agentic systems

This shift marks the transition to Agentic AI Systems.

{ik} INTERVIEW KICKSTART

What You'll be learning today

{ik} INTERVIEW KICKSTART

Today's Agenda

1

Introduction to Gen AI and LLMs

Break

2

Prompt Engineering

Break

3

LLM-Based Agents

Break

4

Q & A

{ik} | INTERVIEW KICKSTART

Now, let’s get started

{ik} | INTERVIEW KICKSTART

Generative AI (Gen AI)

Generating content with massive AI models

{ik} INTERVIEW KICKSTART

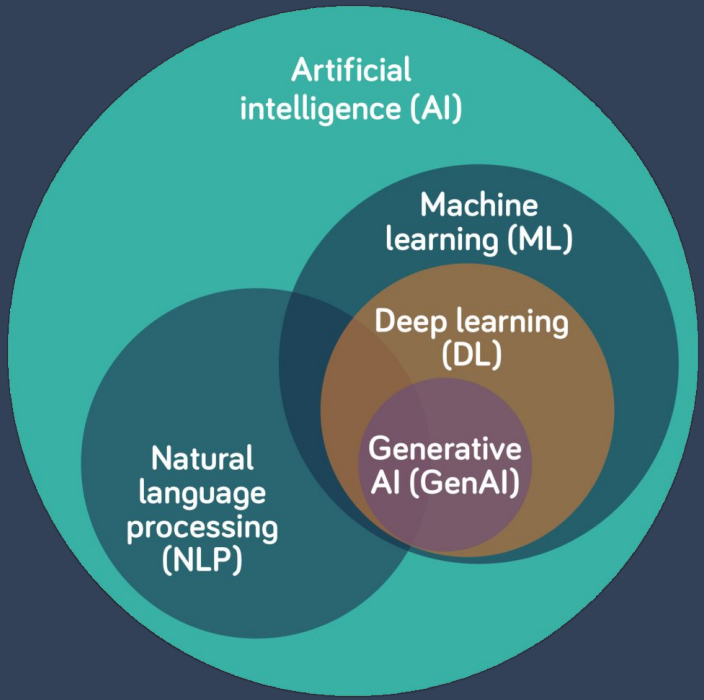
What is Gen AI?



1. Creates **new content** (text, code, images, audio, video)

2. **Understands** natural language (and multimodal) inputs

3. Powered by large **foundation models** trained on massive datasets



Use Cases of Gen AI



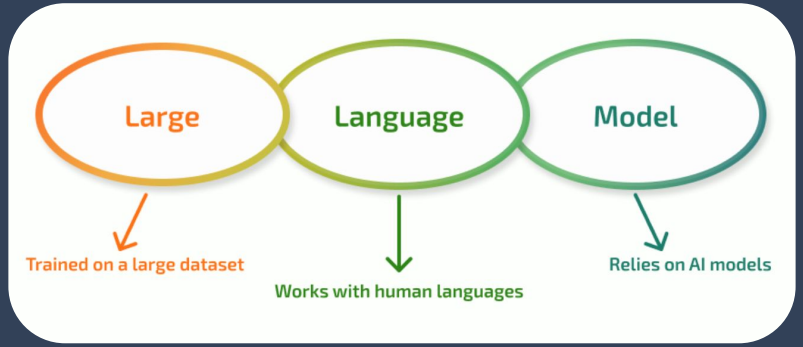
- Content Creation
- Image Generation
- Video Creation
- Code Generation
- Music Generation
- Image Identification
- Idea Generation
- Text Generation
- Task Management
- Data Analytics
- Fraud Detection
- Predictive Analysis
- Risk Management

Behind most **Generative AI** system is a

Large Language Model (LLM)

{ik} | INTERVIEW KICKSTART

What are LLMs?



- Trained on **massive datasets** to learn patterns, meaning, and reasoning
- **Understand and generate** language and other modalities (text, code, images, audio)
- Built on **transformer architectures that scale** to billions of parameters and long context

{ik} | INTERVIEW KICKSTART

Calling LLMs via client libs

{ik} INTERVIEW KICKSTART

If ***LLMs*** are the **engine**,
prompts are how we **steer** them.



{ik} INTERVIEW KICKSTART

Prompt Engineering

- **Context** Set the stage: role, goal, plan etc.

  "You are a senior SRE reviewing a post-mortem"

- **Structure** Use delimiters, Markdown, or XML-like tags

  Use ## Task, ## Input, ## Output headers to organize your prompt

- **Constraints** Output format, tone, edge cases

  Respond in JSON. Max 200 words. Use only the data provided.

- **Iterate** Draft, evaluate, review output, refine.

```
## Role
You are a senior SRE reviewing
a post-mortem.

## Task
Summarize the root cause and
recommend 3 action items.

## Input
{incident report}

## Constraints
- Max 200 words
- Respond in JSON
- Use only the data provided
```

{ik} INTERVIEW KICKSTART

TYPES

ZERO SHOT

No examples provided to LLM to respond to your request. LLM is expected to know the answer.

CHAIN OF THOUGHT

Provide reasoning through intermediate steps to LLMs. Extension of few shot and logical flow.

ONE SHOT

One example or prompt provided to LLM to respond to your request.



COT-SC

Self-Consistency with Chain of Thought: Construct multiple chains of thought, evaluate each one, and ultimately select the most effective and coherent chain.

FEW SHOT

Few examples or prompt provided to LLM to understand what is the expected/ideal response .

TREE OF THOUGHTS

Expand on the chains of thought in a tree format. This allows for backtracking, exploring multiple branches of reasoning stemming from a single root idea.

Limitations of LLMs



No Fresh Knowledge

Trained on a snapshot of the world. Can't access your docs, your databases, or what happened today.



Hallucinations

Confidently wrong. No built-in way to verify or say "I don't know."



No Actions

Generates text, not outcomes. Can't call an API, run a query, or update a ticket.



No Memory

Every conversation starts from zero. No recall of past interactions or accumulated context.

{ik} | INTERVIEW KICKSTART

What if we gave the LLM access to tools and a memory?

{ik} | INTERVIEW KICKSTART

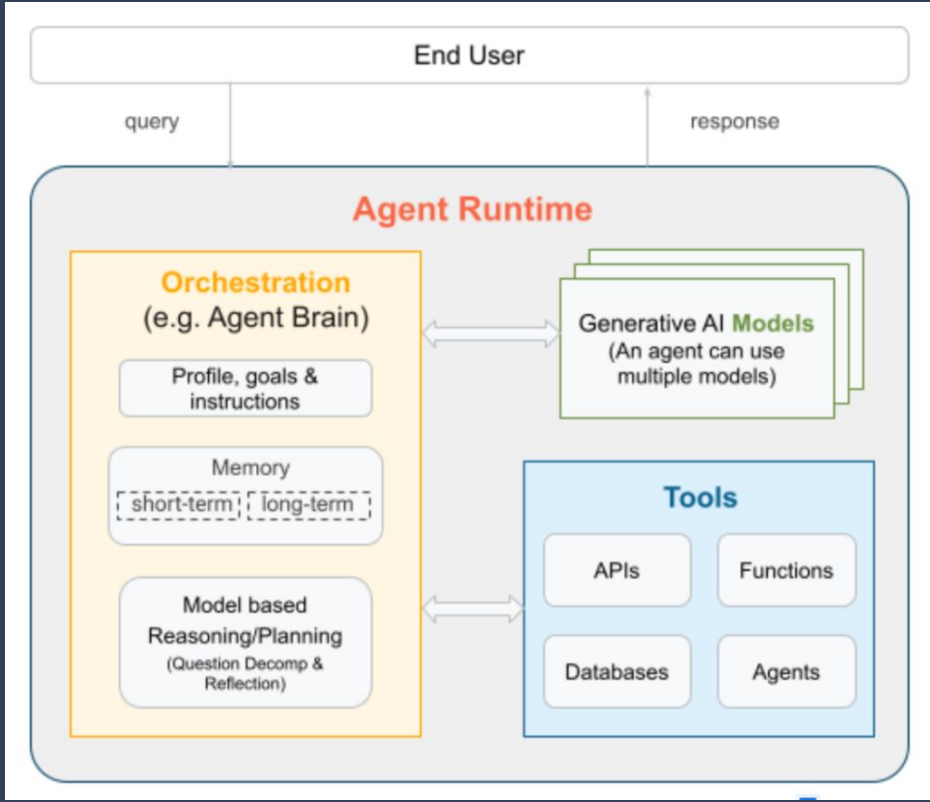
Agent = (Prompt + Tools + Memory) * LLM

An agent allows LLM to **act**, **remember** context, and **decide** what to do next.

Prompt
Defines the behavior

Tools
Enable actions
APIs, search, code execution, databases

Memory
Remembers between turns



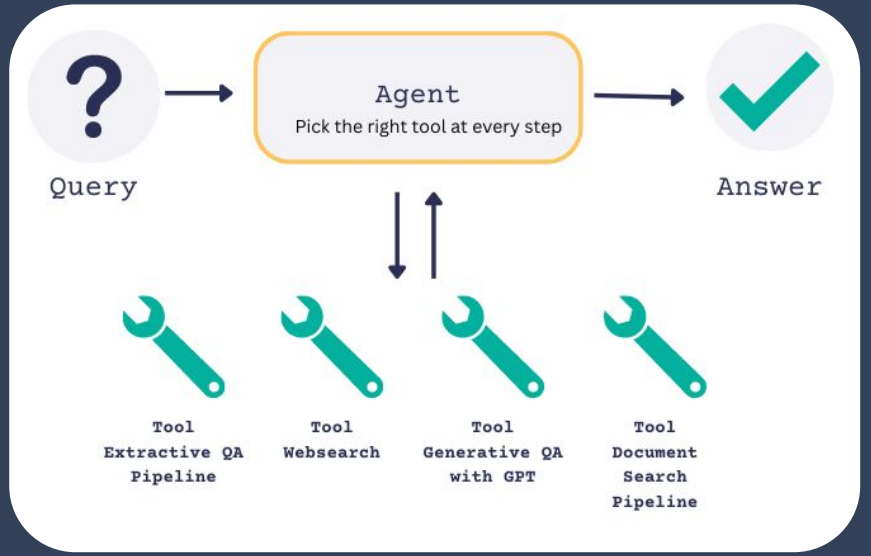
https://neo4j.com/blog/agentic-ai/what-is-ai-agent/

{ik} INTERVIEW KICKSTART

Agents and Tools

Tools turn LLMs from text generators into systems that can act.

- **Retrieve** data from APIs, databases, or vector stores.
- **Execute** actions in external systems, such as sending emails or calling web services.
- Run **deterministic logic**, such as sorting, filtering, data transformation etc.



{ik} INTERVIEW KICKSTART

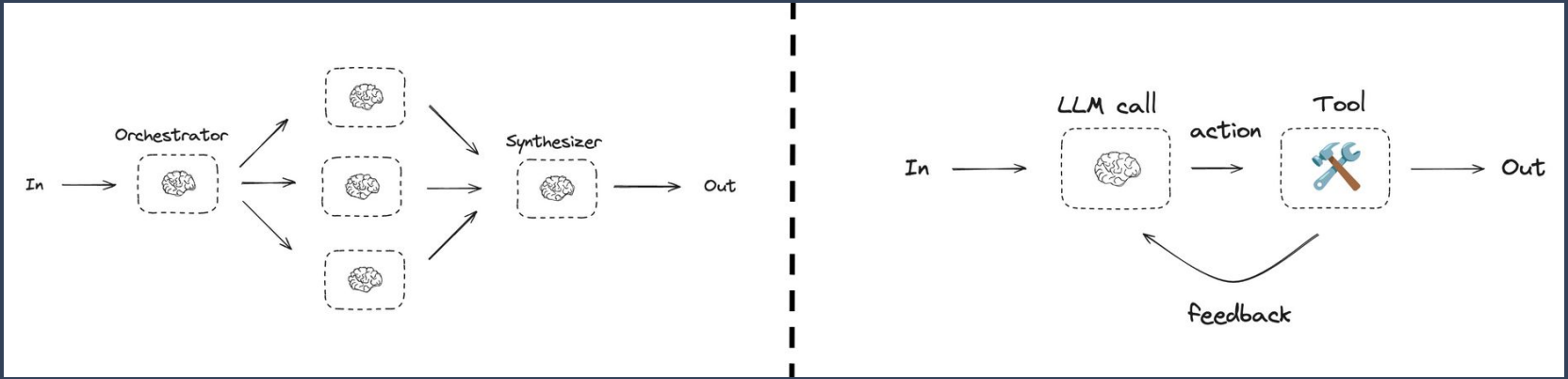
Who's deciding what to do next?

Agentic **Workflows**

LLM & tool control flow is programmatically defined.

Autonomous Agents

The model dynamically decides what to do next.



https://docs.langchain.com/oss/javascript/langgraph/workflows-agents

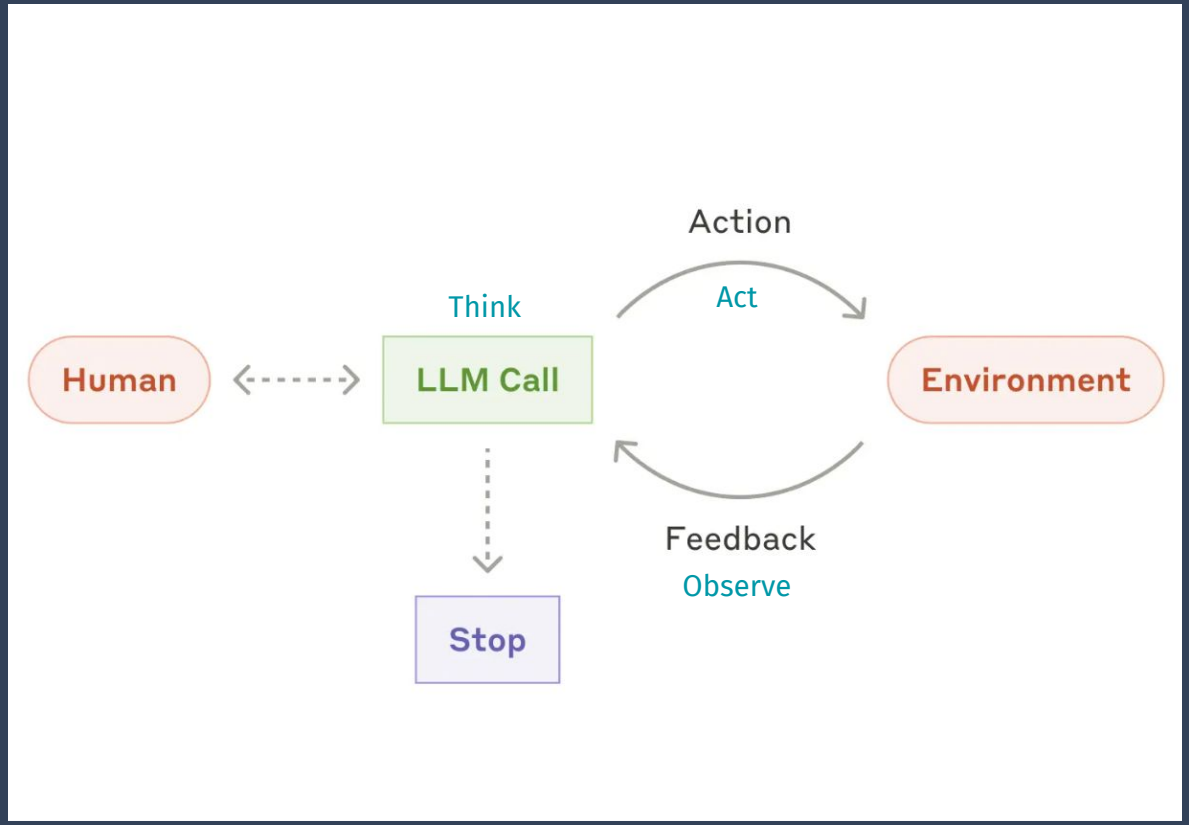
{ik} INTERVIEW KICKSTART

Workflows vs. Autonomous

	🔄 Autonomous	⛓️ Workflow
Orchestration	LLM (dynamic)	Code (fixed pipeline)
Flexibility	High (adapts on the fly)	Low (predefined steps)
Predictability	Lower	Higher
Error handling	LLM reasons about it	Developer codes it
LLM calls	N (unknown ahead of time)	Fixed (known in advance)
Best for	Open-ended tasks	Well-defined processes

💡 **In practice:** Most production systems are hybrids: a workflow defines the high-level stages, but individual stages may use autonomous reasoning within a bounded scope.

The Agent Loop = Loop(Think + Act + Observe)



https://www.anthropic.com/engineering/building-effective-agents

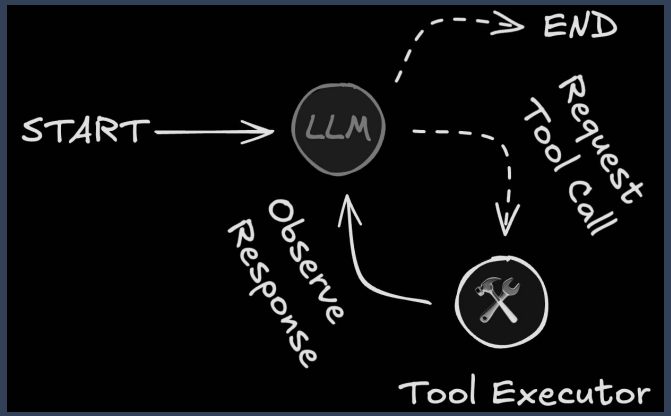
{ik} INTERVIEW KICKSTART

Agent's way of solving problems - Example

Meeting scheduling agent

System prompt
"you are a scheduling assistant helping user book meetings"

Tool definitions
- find_next_free_slot(from_date)
- book_slot(start_time, duration)




```
user -> "book 30 mins on Feb 16"

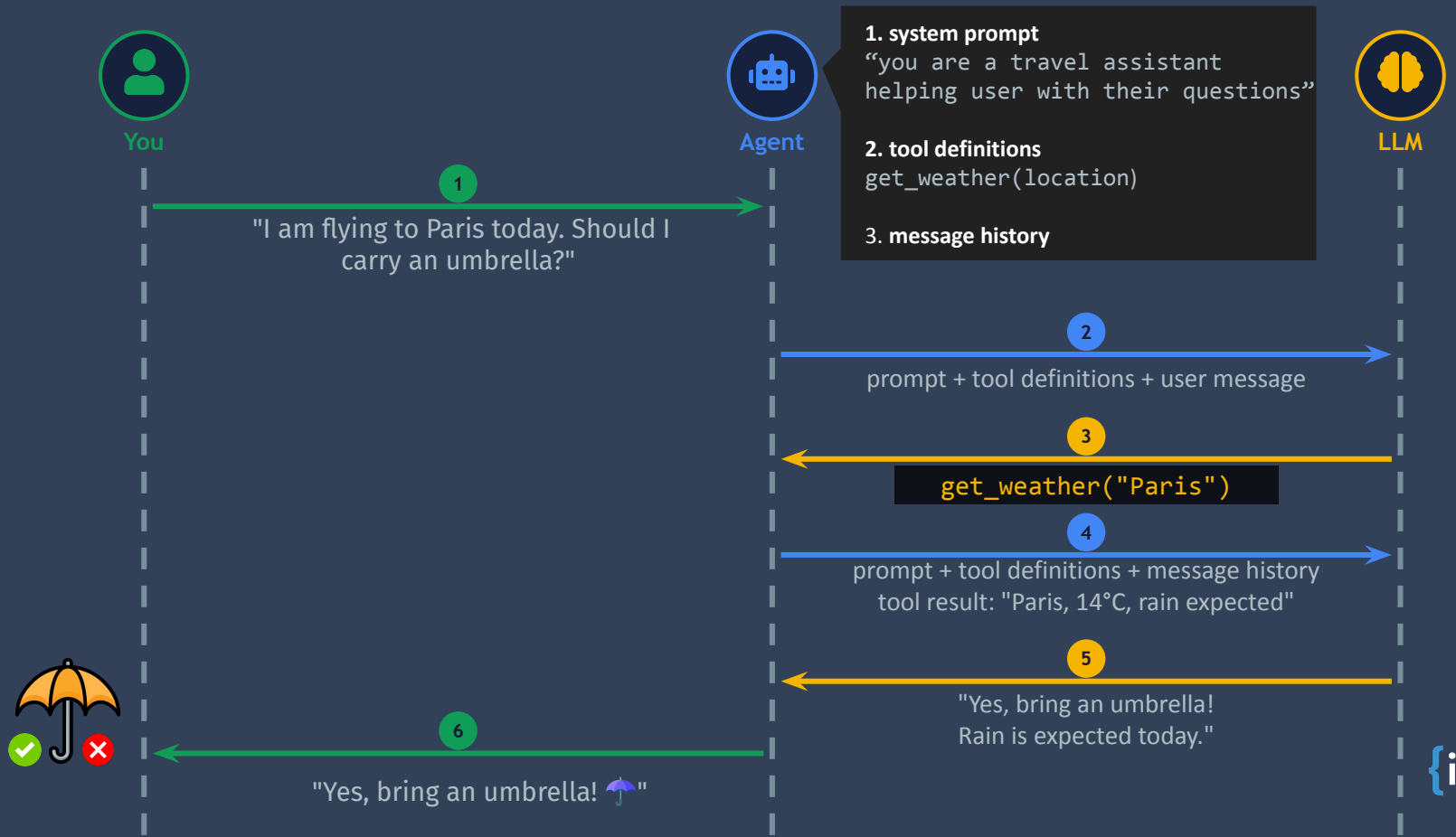
// START

// THINK
1. I need to find next available slot for Feb 16.
// ACT
2. find_next_free_slot("Feb 16") -> 9am
// OBSERVE
3. Okay next available slot is 9am.

// THINK
4. Now I need to book the 9am slot for 30 mins
// ACT
5. book_slot(9am, 30 mins) -> true
// OBSERVE
6. Meeting booked, task is complete.

// END
```

Agent's way of solving problems - Example



{ik} INTERVIEW KICKSTART

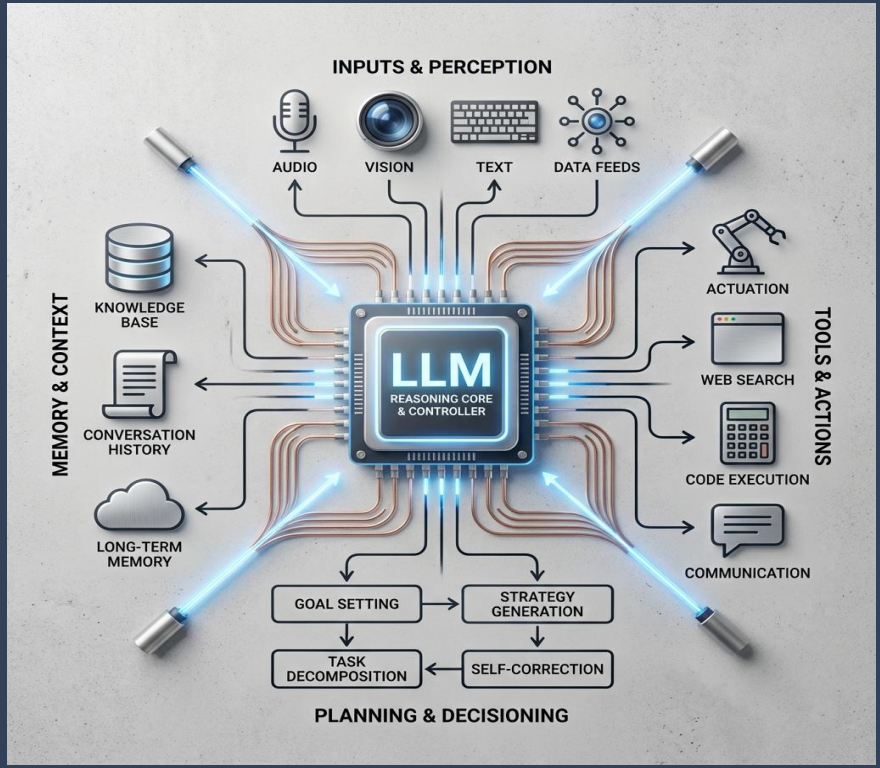
The Cognitive Engine

More Than Just Text

In an agentic system, the LLM stops being a simple chatbot and becomes the **Reasoning Core**.

It acts as the central controller that:

- Parses user intent from ambiguous natural language.
- Decomposes complex problems into steps.
- Synthesizes data returned from those tools.



{ik} INTERVIEW KICKSTART

The ReAct Loop

Reasoning + Acting

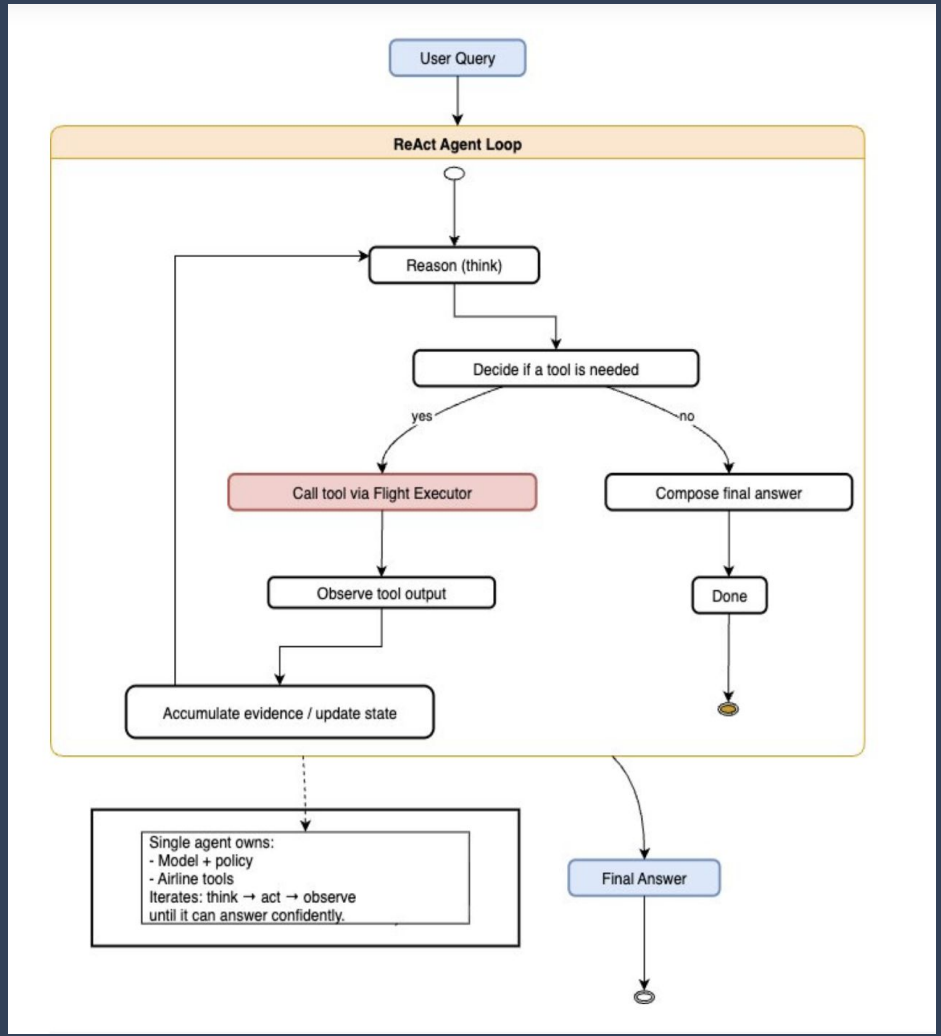
The standard loop for modern agents is **ReAct**:

- **Thought:** The agent analyzes the request.
- **Action:** It decides which tool to call.
- **Observation:** It reads the tool's output.
- **Repeat:** It updates its thought process.

This cycle continues until the final answer is found.

[Customize agent workflows with advanced orchestration techniques using Strands Agents | Artificial Intelligence]

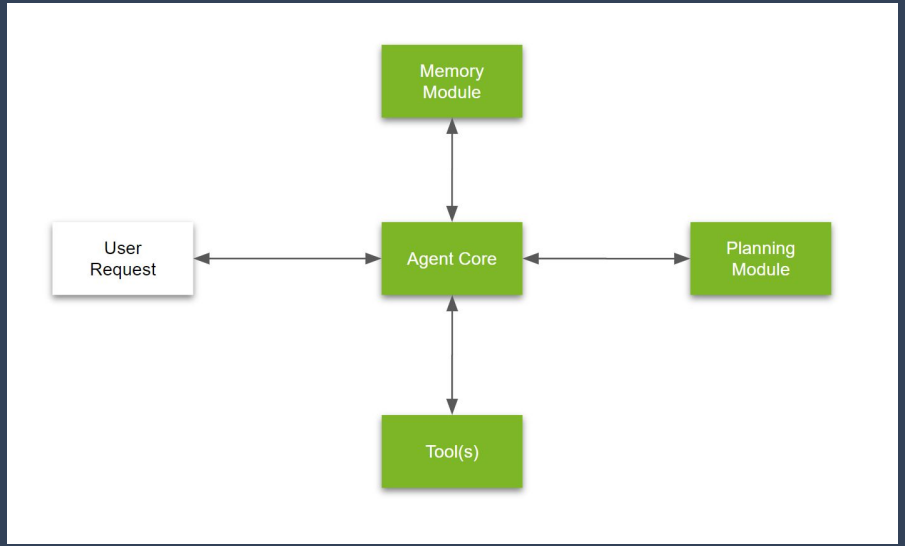
https://arxiv.org/abs/2210.03629



Anatomy of an Agent

A functional agent consists of four main modules:

- **The Brain (LLM):** The decision-maker.
- **Planning:** Breaking down complex goals.
- **Memory:** Storing history and context.
- **Tools:** Capabilities to interact with the world.



https://developer.nvidia.com/blog/introduction-to-llm-agents/

{ik} INTERVIEW KICKSTART

The Three Pillars of Agency



Planning

Decomposing goals into sub-tasks (Chain of Thought).



Memory

Vector Databases allow long-term recall of past events.

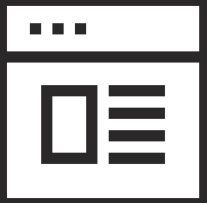


Tools

APIs, Calculators, and Search Engines extend capability.

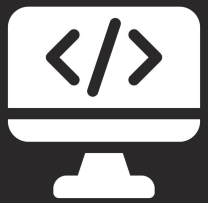
{ik} INTERVIEW KICKSTART

Extending Capabilities with Tools



Web Browsing

Accessing real-time information, news, and live data that is not in the training set.



Code Execution

Writing and running Python scripts to perform complex math or data analysis.



External APIs

Interacting with services like Slack, Jira, or SQL databases to perform actions.

{ik} | INTERVIEW KICKSTART

Memory: Short vs. Long Term

Short-Term (Context)

The "RAM" of the Agent.

This is limited by the model's context window (e.g., 128k tokens). It holds the immediate conversation history, the current prompt, and recent tool outputs.

Limitation: Cleared when the session ends.

Long-Term (Vector Store)

The "Hard Drive" of the Agent.

Uses Vector Databases (RAG) to store vast amounts of information. The agent retrieves only relevant "memories" based on semantic similarity to the current task.

Benefit: Infinite persistence.

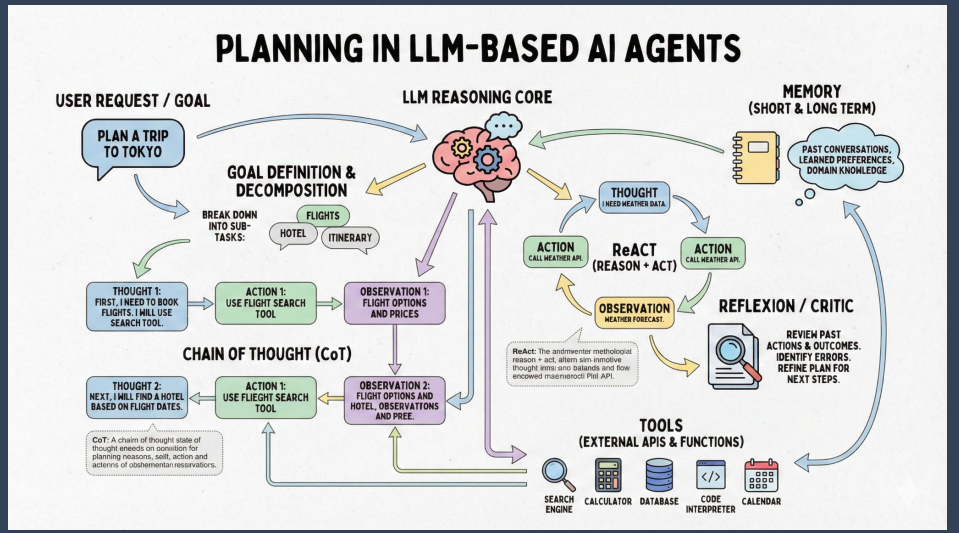
{ik} | INTERVIEW KICKSTART

Planning & Reasoning

Breaking Down Complexity

Agents cannot solve complex tasks in one shot. They use planning techniques to improve reliability.

- **Chain of Thought:** Stepping through logic explicitly.
- **Tree of Thoughts:** Exploring multiple possibilities before committing.
- **Reflection:** Reviewing past actions to catch errors.



{ik} INTERVIEW KICKSTART

The Future: Multi-Agent Systems



Specialization

Instead of one generalist, swarms of specialized agents (Coder, Designer, Writer) will collaborate.



Orchestration

"Manager" agents will break down huge projects and assign tasks to "worker" agents.



Autonomy

Agents will run continuously in the background, optimizing workflows without human input.

{ik} | INTERVIEW KICKSTART

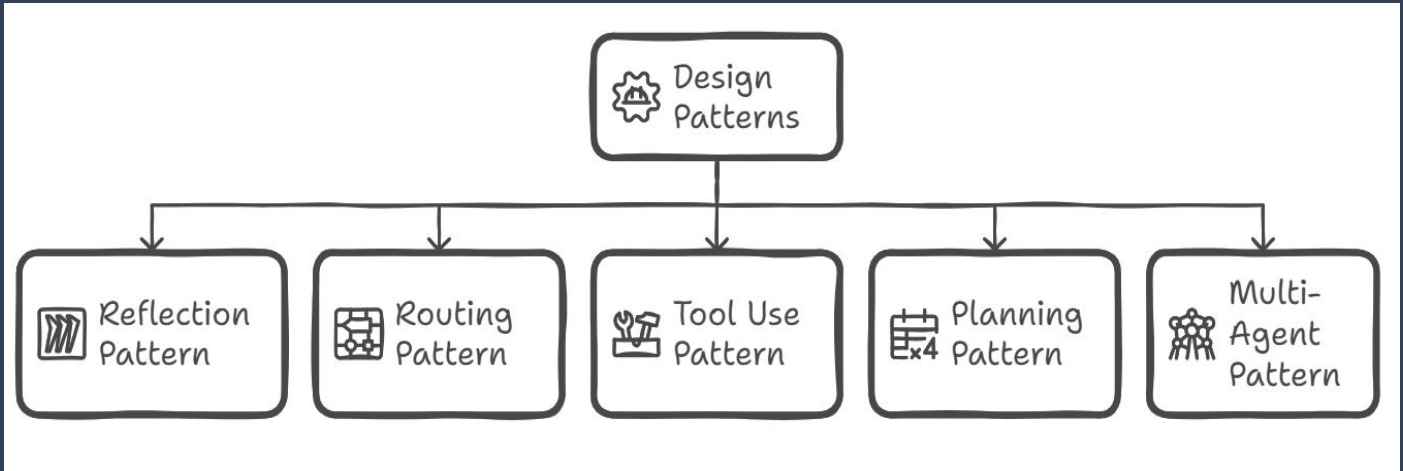
You've learned what agents can do.

Now let's learn how to design them for real-world applications.

{ik} INTERVIEW KICKSTART

Agentic design Pattern

- Agentic Design Patterns are the building blocks for LLM systems that go beyond single prompt–response interactions.
- They use composable structures to orchestrate reasoning, decisions, tools, and control flow.
- By replacing hard-coded logic with system structure, these patterns enable reliable adaptation and scaling—from simple assistants to autonomous, goal-driven agents.



{ik} INTERVIEW KICKSTART

Let’s build a CRM Lead qualifier agent

This README documents the complete agent design and routing logic for the CRM Lead Qualifier Agent

{ik} INTERVIEW KICKSTART



Feedback Poll for Instructor

{ik} | INTERVIEW KICKSTART

Questions?

Exploring the frontier of autonomous AI.

{ik} INTERVIEW KICKSTART

For Instructors - Encourage Learner Interaction

- Unmute learners and invite them to ask questions verbally
- Promote open discussion instead of only chat-based Q&A
- Actively engage learners to maintain interactivity throughout the session

{ik} INTERVIEW KICKSTART

After building your agent, package the project with proper documentation (README) and publish it on GitHub to showcase it in your portfolio.

README Template for Agentic 2.0

{ik} INTERVIEW KICKSTART

Build → Document → Publish (Every Agent)

For every agent built in this course:

- Package the project with a clear, structured README
- Upload it to your GitHub portfolio
- Treat each agent as a standalone, portfolio-ready artifact

This workflow will be applied consistently in every class as agents are built and published.

{ik} INTERVIEW KICKSTART

Week 0 - Mini Project

{ik} INTERVIEW KICKSTART

Mini Project: Hello Agent – CSV FAQ Agent

The Challenge with Manual CSV Queries:

- Many organizations store key knowledge in CSV files (FAQs, policies, docs).
- Teams manually search these files during live conversations.
- **This leads to:**
  - Slow responses — information is hard to query quickly
  - Inconsistent answers — different team members may interpret differently
  - Inefficient workflows — time wasted on repetitive searches



{ik} INTERVIEW KICKSTART

Mini Project: Hello Agent – CSV FAQ Agent

CSV FAQ Agent Requirements:

- **CSV Upload:** Users can upload one or more CSV files.
- **Natural Language Queries:** Users can ask questions in plain English.
- **Data-Driven Answers:** Responses are strictly based on the uploaded CSV data.
- **Missing Data Handling:** Clearly indicate when an answer is not found in the data.

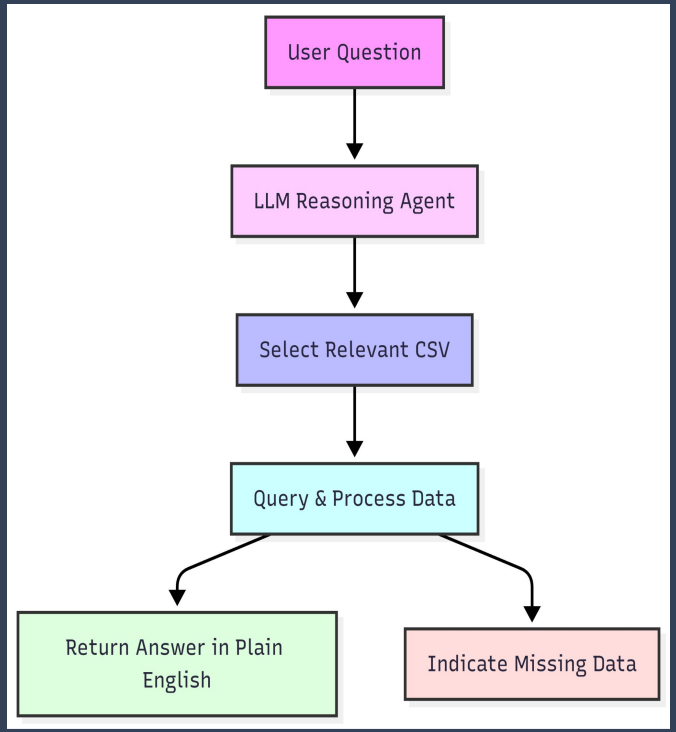
{ik} INTERVIEW KICKSTART

Hello Agent – CSV FAQ Agent: Solution

To solve the problem efficiently and meet the requirements:

- **Load all CSV files** into memory for access.
- **Initialize an LLM agent** (ChatOpenAI, GPT-4o-mini) to reason over data.
- **Connect multiple CSVs** to the agent for multi-source decision making.
- **Design system prompts** to guide the agent: pick the correct CSV, answer only from data, return plain English responses.
- **Implement a chat loop:** user question → agent queries → returns answer or indicates missing data.

Let's now look at the Solution



{ik} INTERVIEW KICKSTART



{ik} | INTERVIEW KICKSTART